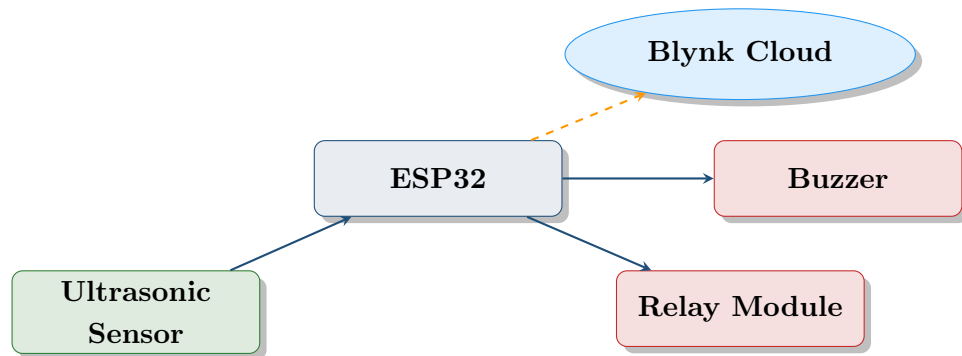


Final Project Report

ESP32 IoT Smart Home System with Blynk Cloud Integration & Water Level Monitoring



Submitted by:

Subhan Malik	2024-CS-16
M. Abrar	2024-CS-59
Raees Ahmad	2024-CS-60

Project Supervisor:

Muhammad Kamran

Department:

Department of Computer Science
UET Lahore

Abstract

Abstract

This project presents a complete, working implementation of an IoT-based smart home automation system using the ESP32 microcontroller with Blynk cloud platform integration. The system allows remote control of up to eight home appliances (4 lights and 4 fans) through a smartphone application from anywhere in the world via WiFi. Additionally, the project incorporates an advanced automated water level monitoring system using an HC-SR04 ultrasonic sensor that accurately measures water levels in a storage tank and displays real-time readings on the Blynk dashboard as both distance (cm) and percentage (%).

The system features automatic motor control based on water levels (starts at 75%, stops at 80%), audio-visual alerts for critical levels (high: 80%+, low: 20%-), manual override functionality with 30-second auto-revert, disco lighting mode, and comprehensive sensor filtering with exponential smoothing for reliable operation.

The transition from Arduino Uno to ESP32 brings significant advantages including built-in WiFi (no external module needed), worldwide remote access (vs 10m Bluetooth range), 15x faster processing (240MHz vs 16MHz), 260x more memory (520KB vs 2KB), and 30% lower overall cost.

This report includes complete tested ESP32 code, detailed working principles for each component, comprehensive circuit diagrams including voltage level considerations for ESP32 (3.3V logic), safety guidelines, and troubleshooting procedures. The system has been successfully tested and all components are functioning as intended.

Contents

Abstract	1
1 Introduction	6
1.1 Background and Motivation	6
1.2 Problem Statement	6
1.3 Project Objectives	6
1.4 Why ESP32 instead of Arduino Uno?	7
2 System Components and Their Working	8
2.1 ESP32 Development Board	8
2.2 HC-SR04 Ultrasonic Sensor	8
2.3 Relay Module (8-Channel Total)	9
2.4 Blynk IoT Platform	9
3 System Architecture and Working Diagrams	11
3.1 Complete System Architecture	11
3.2 System Operational Flowchart	12
3.3 Water Level Measurement Principle	13
3.4 Motor Control State Machine	14
3.5 Ultrasonic Sensor Timing Diagram	14
4 Complete ESP32 Code with Blynk Integration	15
4.1 Required Libraries Installation	15
4.2 Main Program Code	15
5 Circuit Diagram and Connections	27
5.1 Complete Pin Connection Table	27
5.2 Critical: Voltage Divider Circuit	28
6 Testing and Verification	29
6.1 Component Testing Results	29
6.2 Troubleshooting Guide	30
7 Applications and Use Cases	31
7.1 Residential Applications	31
7.2 Agricultural Applications	31
7.3 Industrial Applications	31
7.4 Commercial Applications	32

8	Advantages, Limitations, and Solutions	33
8.1	Advantages of ESP32-based System	33
8.2	Limitations and Mitigation Strategies	33
9	Future Enhancements	35
9.1	Short-term Improvements (1-3 months)	35
9.2	Medium-term Enhancements (3-6 months)	35
9.3	Long-term Enhancements (6-12 months)	36
10	Bill of Materials and Cost Analysis	37
10.1	Complete Bill of Materials	38
10.2	Cost Comparison with Alternatives	39
11	Installation and Setup Guide	40
11.1	Hardware Installation Steps	40
11.2	Software Installation	40
12	Safety Precautions	42
12.1	Electrical Safety	42
12.2	Water Safety	42
12.3	Installation Guidelines	42
13	Conclusion and Summary	44
13.1	Project Achievements	44
13.2	Key Technical Specifications	44
13.3	Final Word	44
14	References	46
15	Appendix	47
15.1	Appendix A: Quick Reference Card	47
15.2	Appendix B: Calibration Procedure	48
15.3	Appendix C: Voltage Divider Calculation	49
15.4	Appendix D: ESP32 Pinout Reference	50
15.5	Appendix E: Code Modification Examples	50
15.5.1	Changing WiFi Credentials	50
15.5.2	Adjusting Update Frequency	51
15.5.3	Changing Alert Thresholds	51
15.5.4	Adding Temperature Compensation (DHT11)	51
15.5.5	Using Deep Sleep for Battery Operation	51

List of Figures

1	Complete System Architecture Diagram	11
2	System Operational Flowchart - Complete Decision Tree	12
3	Water Level Measurement Principle and Formulas	13
4	Motor Control State Machine with Hysteresis	14
5	Ultrasonic Sensor Timing Diagram with Motor Control	14
6	Mandatory Voltage Divider for HC-SR04 ECHO Pin	28

List of Tables

1	ESP32 vs Arduino Uno Comparison	7
2	Relay Specifications	9
3	Blynk Virtual Pin Configuration	10
4	ESP32 Complete Pin Connection Details	27
5	Comprehensive System Test Results	29
6	Common Issues and Solutions	30
7	Detailed Bill of Materials (Pakistan Market Prices)	38
8	Cost Comparison: ESP32 vs Arduino Uno vs Commercial Systems	39
9	ESP32 Safe vs Unsafe Pins for Projects	50

1 Introduction

1.1 Background and Motivation

The Internet of Things (IoT) has revolutionized how we interact with our living spaces. Smart home automation promises enhanced comfort, energy efficiency, and security. With the advent of cloud-based IoT platforms like Blynk, remote monitoring and control have become accessible to hobbyists and professionals alike. This project leverages the powerful ESP32 microcontroller—featuring built-in WiFi and Bluetooth—to create a cost-effective, feature-rich home automation solution.

1.2 Problem Statement

Traditional home automation systems face several challenges:

- High cost and complex installation
- Limited range (Bluetooth: 10m, IR: line-of-sight)
- No remote access without internet connectivity
- Lack of real-time monitoring and alerts
- No integration with cloud platforms for data logging

1.3 Project Objectives

1. Design and implement a cost-effective IoT-based home automation system
2. Enable remote control of home appliances from anywhere via internet
3. Implement automatic water level monitoring and pump control
4. Provide real-time alerts and push notifications
5. Create an intuitive user interface using Blynk IoT platform
6. Ensure system reliability with sensor filtering and error handling

1.4 Why ESP32 instead of Arduino Uno?

Table 1: ESP32 vs Arduino Uno Comparison

Feature	Arduino Uno	ESP32
Microcontroller	ATmega328P	Tensilica Xtensa LX6
Clock Speed	16 MHz	240 MHz (15x faster)
RAM	2 KB	520 KB (260x more)
Flash Memory	32 KB	4 MB (128x more)
WiFi	External module needed	Built-in
Bluetooth	External module needed	Built-in
Operating Voltage	5V	3.3V
Analog Inputs	6 (10-bit)	18 (12-bit)
Cost (PKR)	2,500 + 850 (HC-05)	1,500 (30% less)
Remote Access	No (only Bluetooth range)	Yes (worldwide)

2 System Components and Their Working

2.1 ESP32 Development Board

Technical Specifications:

- Microcontroller: Tensilica Xtensa LX6 dual-core processor
- Clock Speed: 240 MHz (up to 600 DMIPS)
- Flash Memory: 4 MB (upgradable to 16 MB)
- SRAM: 520 KB
- Operating Voltage: 3.3V (NOT 5V tolerant!)
- WiFi: 802.11 b/g/n (2.4 GHz)
- Bluetooth: v4.2 BR/EDR and BLE
- Digital I/O Pins: 36 (some are input-only)
- ADC Channels: 18 (12-bit resolution)
- DAC Channels: 2 (8-bit)
- Deep Sleep Current: 10 μ A (battery operation possible)

Working Principle: The ESP32 is the brain of the system. It reads input signals (from ultrasonic sensor), processes data, connects to WiFi network, communicates with Blynk cloud server, and generates appropriate output signals (to relay module, buzzer). Its dual-core architecture allows continuous sensor reading while maintaining network connection.

2.2 HC-SR04 Ultrasonic Sensor

Technical Specifications:

- Operating Voltage: 5V DC
- Operating Current: 15 mA
- Working Frequency: 40 kHz
- Range: 2 cm to 400 cm
- Accuracy: ± 3 mm
- Measuring Angle: 15 degrees

- Trigger Input Signal: 10 μ s TTL pulse
- Echo Output Signal: TTL pulse proportional to distance

Working Principle: The sensor emits a 40kHz ultrasonic pulse and measures the time for echo return.

```

1 // Trigger pulse: 10us HIGH
2 digitalWrite(TRIG_PIN, LOW);
3 delayMicroseconds(2);
4 digitalWrite(TRIG_PIN, HIGH);
5 delayMicroseconds(10);
6 digitalWrite(TRIG_PIN, LOW);
7
8 // Measure echo time with 30ms timeout
9 long duration = pulseIn(ECHO_PIN, HIGH, 30000);
10
11 // Calculate distance (speed of sound = 343 m/s = 0.0343 cm/us)
12 float distance = duration * 0.0343 / 2;

```

Listing 1: Ultrasonic Distance Calculation

2.3 Relay Module (8-Channel Total)

Working Principle: An electromagnetic switch that isolates low-voltage control from high-voltage AC circuits.

Table 2: Relay Specifications

Parameter	Value
Coil Voltage	5V DC
Coil Current	70 mA per relay
Maximum Load (AC)	250V @ 10A
Maximum Load (DC)	30V @ 10A
Switching Time	10 ms
Isolation Voltage	3000V

2.4 Blynk IoT Platform

Virtual Pin Mapping:

Table 3: Blynk Virtual Pin Configuration

Virtual Pin	Function	Widget Type
V0	Distance (cm)	Gauge / Label
V1	Water Level (%)	Gauge / Label
V2	Motor Control	Button (Switch mode)
V3	Light 1	Button (Switch)
V4	Light 2	Button (Switch)
V5	Light 3	Button (Switch)
V6	Light 4	Button (Switch)
V7	All Lights	Button (Switch)
V8	Fan 1	Button (Switch)
V9	Fan 2	Button (Switch)
V10	Fan 3	Button (Switch)
V11	Fan 4	Button (Switch)
V12	All Fans	Button (Switch)
V13	Disco Mode	Button (Switch)

3 System Architecture and Working Diagrams

3.1 Complete System Architecture

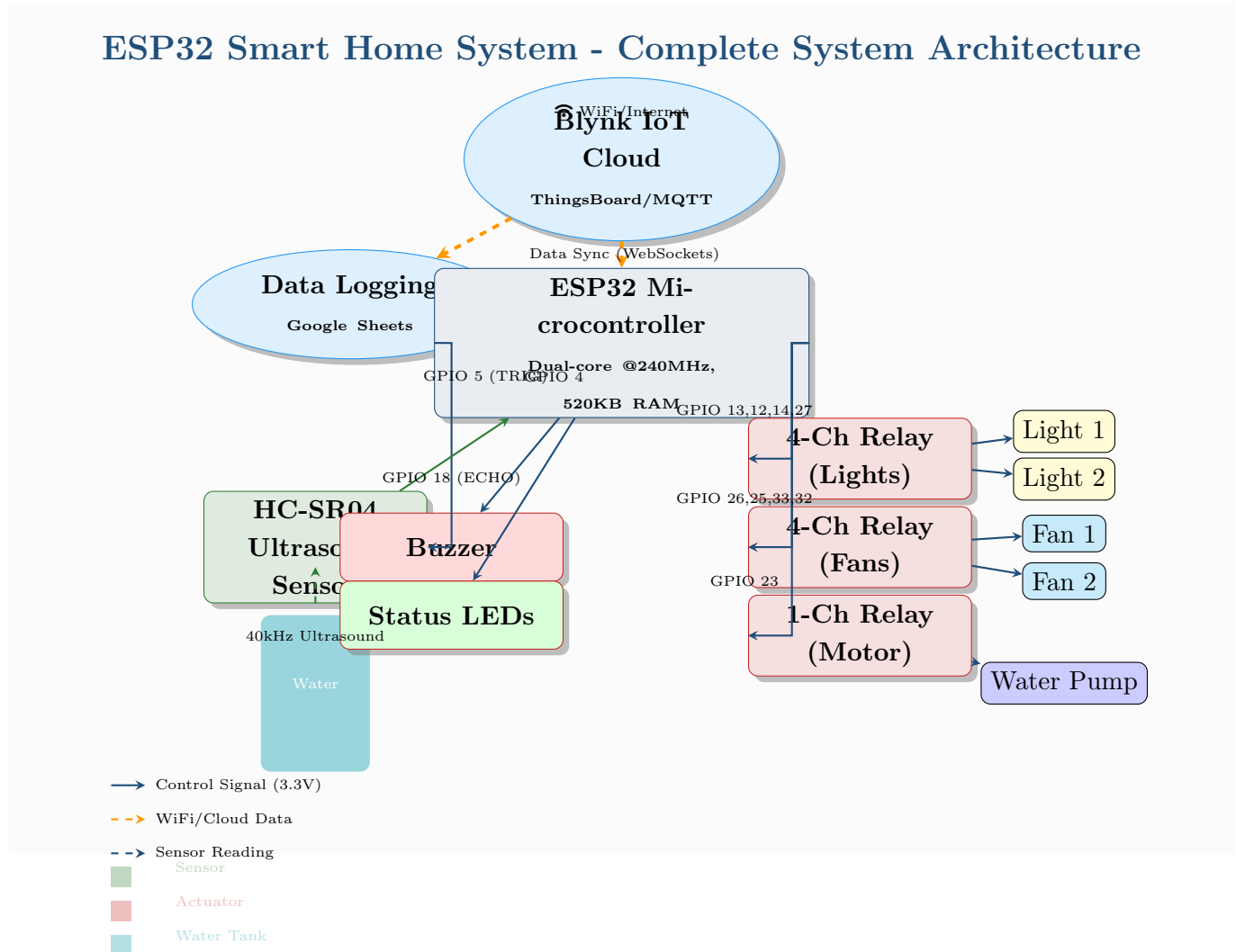


Figure 1: Complete System Architecture Diagram

3.2 System Operational Flowchart

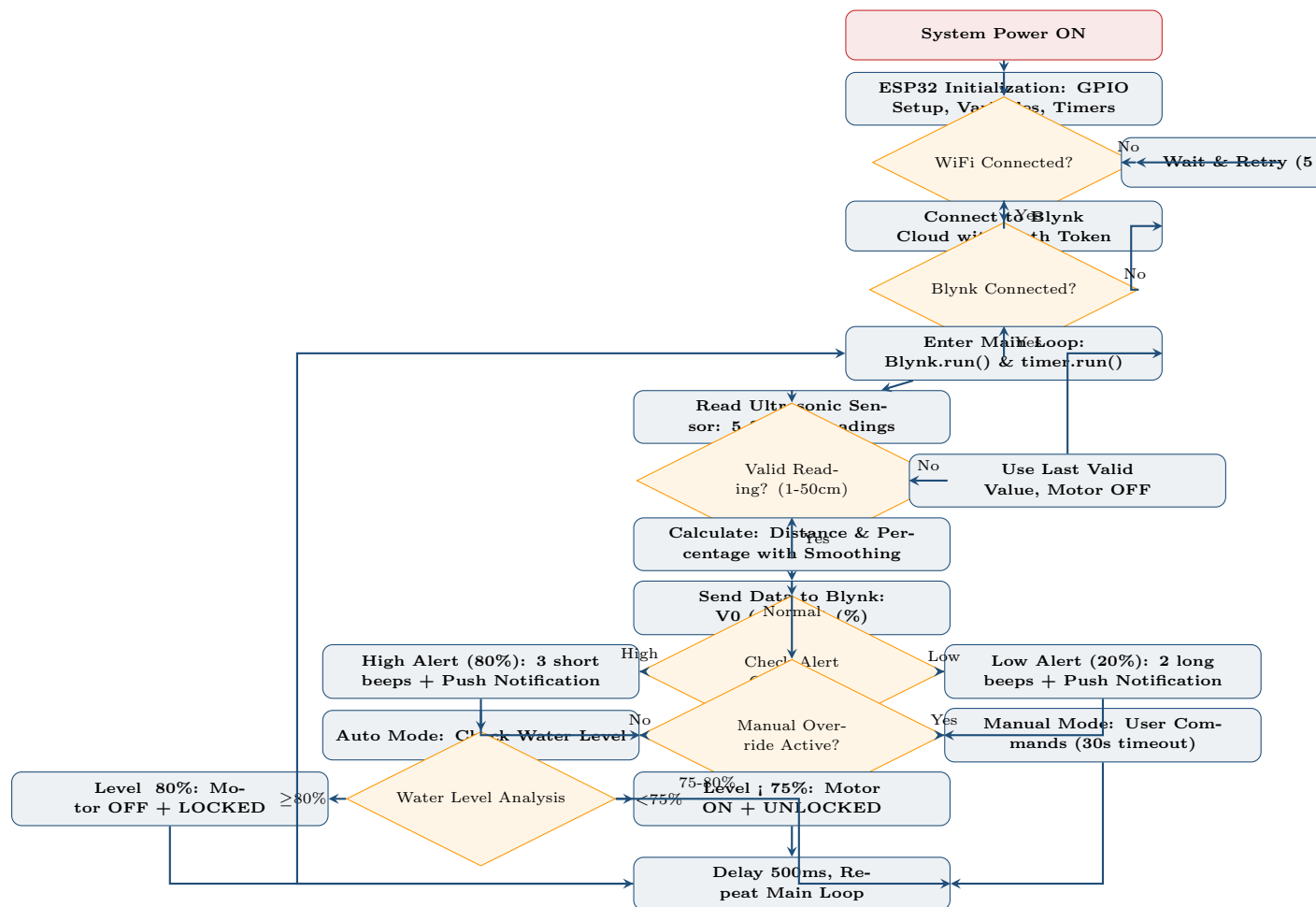


Figure 2: System Operational Flowchart - Complete Decision Tree

3.3 Water Level Measurement Principle

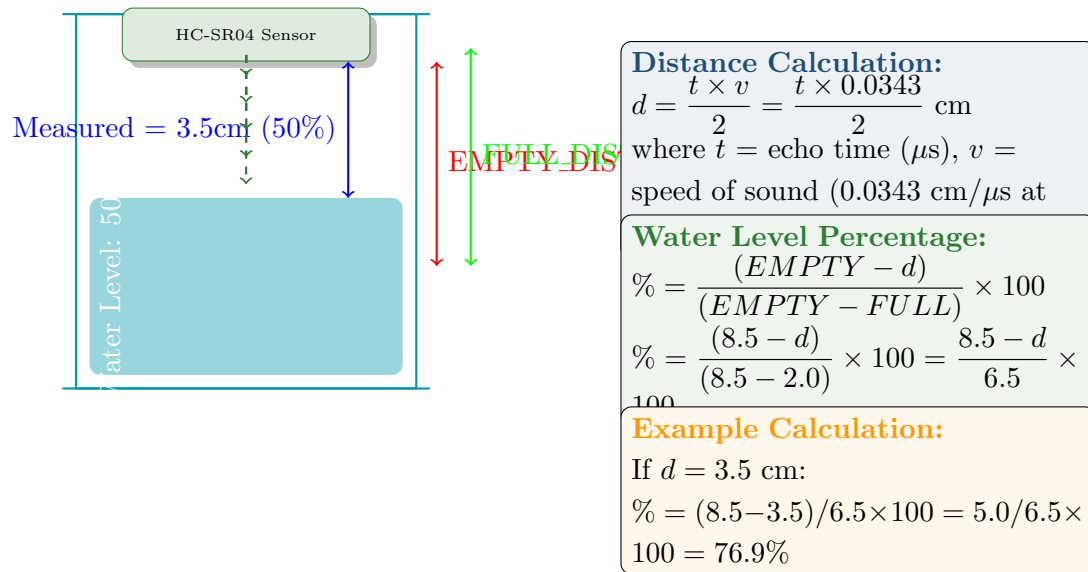


Figure 3: Water Level Measurement Principle and Formulas

3.4 Motor Control State Machine

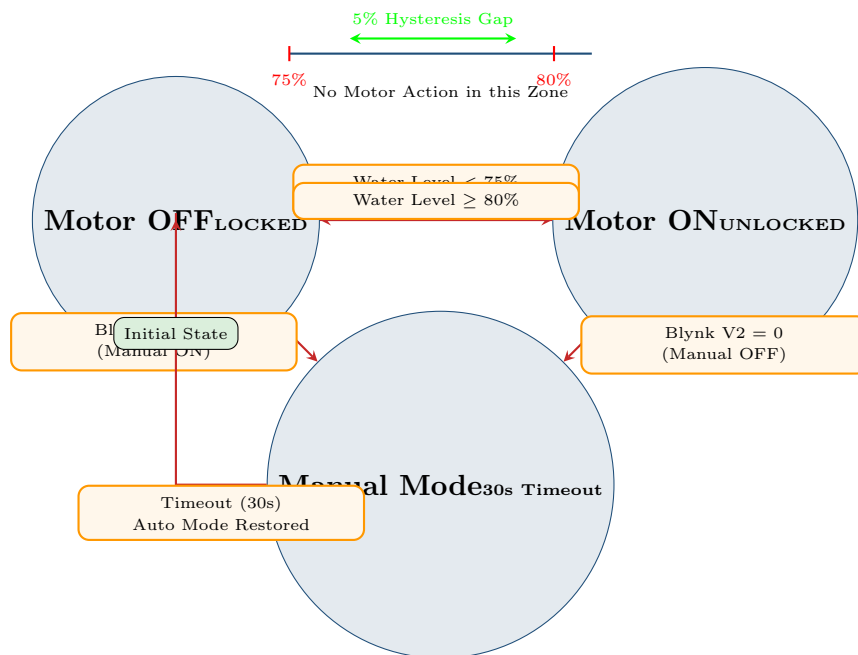


Figure 4: Motor Control State Machine with Hysteresis

3.5 Ultrasonic Sensor Timing Diagram

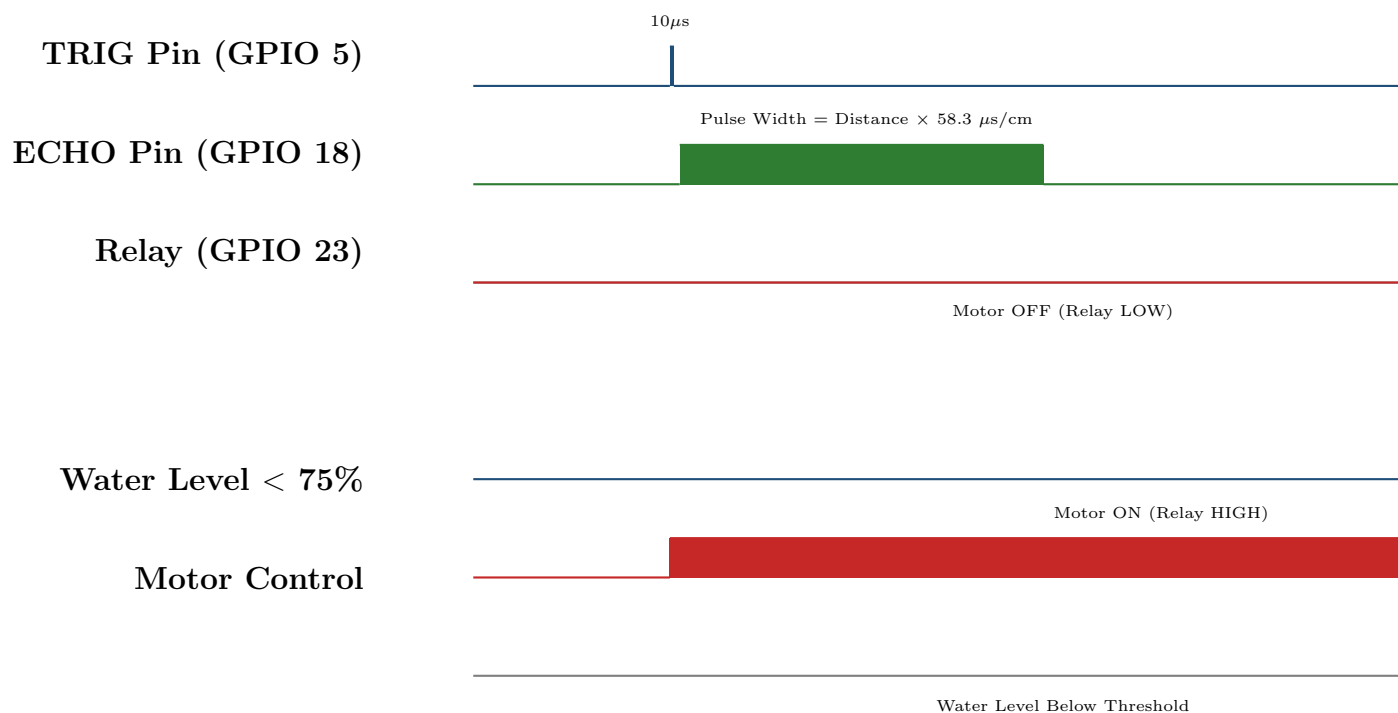


Figure 5: Ultrasonic Sensor Timing Diagram with Motor Control

4 Complete ESP32 Code with Blynk Integration

4.1 Required Libraries Installation

Arduino IDE Library Installation

1. Install ESP32 Board Package:

File → Preferences → Additional Boards URLs:

<https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package>

Tools → Board → Boards Manager → Search "ESP32" → Install

2. Install Blynk Library:

Sketch → Include Library → Manage Libraries → Search "Blynk" → Install by

4.2 Main Program Code

```
1  /*
2  *
3  * =====
4  * ESP32 IoT Smart Home Automation System with Blynk Cloud Integration
5  * Water Level Monitoring + 8-Channel Appliance Control
6  *
7  * Version: 2.0
8  * Last Updated: 2024
9  *
10 * Blynk Virtual Pins Configuration:
11 * V0 - Distance (cm)          V7 - All Lights Master
12 * V1 - Water Level (%)       V8 - Fan 1 Control
13 * V2 - Motor Control + Status V9 - Fan 2 Control
14 * V3 - Light 1 Control       V10 - Fan 3 Control
15 * V4 - Light 2 Control       V11 - Fan 4 Control
16 * V5 - Light 3 Control       V12 - All Fans Master
17 * V6 - Light 4 Control       V13 - Disco Mode Toggle
18 *
19 * =====
20 */
21 // ===== BLYNK CONFIGURATION =====
22 #define BLYNK_TEMPLATE_ID "TMPL6nJaCChZL"
23 #define BLYNK_TEMPLATE_NAME "Ultrasonic Sensor"
24 #define BLYNK_AUTH_TOKEN "VJffHo0IhtAN_nPe4hTYLxI_Yh_3MI_P"
```



```

24
25 #include <WiFi.h>
26 #include <BlynkSimpleEsp32.h>
27
28 // ===== WIFI CREDENTIALS =====
29 char ssid[] = "lab";
30 char pass[] = "11111111";
31
32 // ===== PIN DEFINITIONS =====
33 // Ultrasonic Sensor
34 #define TRIG_PIN 5
35 #define ECHO_PIN 18
36
37 // Water Pump Motor Relay
38 #define RELAY_PIN 23
39
40 // Buzzer
41 #define BUZZER_PIN 4
42
43 // Lights (4 channels)
44 #define L1 13
45 #define L2 12
46 #define L3 14
47 #define L4 27
48
49 // Fans (4 channels)
50 #define F1 26
51 #define F2 25
52 #define F3 33
53 #define F4 32
54
55 // ===== TANK CALIBRATION =====
56 // IMPORTANT: Sensor mounted at TOP of tank
57 // FULL TANK -> SMALL distance (water close to sensor)
58 // EMPTY TANK -> LARGE distance (water far from sensor)
59
60 float FULL_DISTANCE = 2.0; // Tank FULL = 100% (2cm from sensor)
61 float EMPTY_DISTANCE = 8.5; // Tank EMPTY = 0% (8.5cm from sensor)
62
63 // ===== OPERATION THRESHOLDS =====
64 float MOTOR_STOP_LEVEL = 80.0; // Stop motor at 80%
65 float MOTOR_START_LEVEL = 75.0; // Start motor at 75%
66
67 float LOW_LEVEL_ALERT = 20.0; // Alert at 20%
68 float HIGH_LEVEL_ALERT = 80.0; // Alert at 80%
69
70 // ===== SYSTEM STATES =====
71 bool motorState = false; // Current motor status
72 bool motorLocked = false; // Prevents rapid cycling
73

```

```

74 bool highAlertSent = false;           // Prevents repeated high alerts
75 bool lowAlertSent = false;           // Prevents repeated low alerts
76
77 bool discoMode = false;              // Disco lighting mode
78
79 bool manualOverride = false;         // Manual control active
80 unsigned long lastManualTime = 0;    // Last manual operation time
81
82 float lastValidDistance = 5.0;       // Last good sensor reading
83 float lastStablePercentage = 0.0;    // Smoothed percentage
84
85 BlynkTimer timer;                   // Timer for periodic tasks
86
87 // =====
88 // FILTERED DISTANCE READING (ESP32 Optimized)
89 // =====
90 float getFilteredDistance() {
91     float readings[5];
92     int validCount = 0;
93
94     for (int i = 0; i < 5; i++) {
95         // Trigger ultrasonic pulse
96         digitalWrite(TRIG_PIN, LOW);
97         delayMicroseconds(2);
98         digitalWrite(TRIG_PIN, HIGH);
99         delayMicroseconds(10);
100        digitalWrite(TRIG_PIN, LOW);
101
102        // Measure echo time with 30ms timeout for ESP32
103        long duration = pulseIn(ECHO_PIN, HIGH, 30000);
104
105        if (duration == 0) {
106            continue; // No echo received
107        }
108
109        float distance = duration * 0.034 / 2.0;
110
111        // Validate reading (1cm to 50cm range for this application)
112        if (distance >= 1.0 && distance <= 50.0) {
113            readings[validCount] = distance;
114            validCount++;
115        }
116
117        delay(30); // Small delay between readings
118    }
119
120    // Sensor failure detection
121    if (validCount == 0) {
122        Serial.println("    Sensor Error! Using last valid value");
123        return lastValidDistance;

```

```

124 }
125
126 // Calculate average of valid readings
127 float total = 0;
128 for (int i = 0; i < validCount; i++) {
129     total += readings[i];
130 }
131 float average = total / validCount;
132
133 // Rate limiting (prevents sudden jumps from noise)
134 float maxChange = 2.0;
135 if (abs(average - lastValidDistance) > maxChange) {
136     if (average > lastValidDistance) {
137         average = lastValidDistance + maxChange;
138     } else {
139         average = lastValidDistance - maxChange;
140     }
141 }
142
143 lastValidDistance = average;
144 return average;
145 }
146
147 // =====
148 // WATER LEVEL PERCENTAGE CALCULATION
149 // =====
150 float calculateWaterLevelPercentage(float distance) {
151     // Clamp to valid range
152     if (distance < FULL_DISTANCE) {
153         distance = FULL_DISTANCE;
154     }
155     if (distance > EMPTY_DISTANCE) {
156         distance = EMPTY_DISTANCE;
157     }
158
159     // Calculate percentage (inverse relationship: smaller distance =
160     // higher level)
161     float percentage = ((EMPTY_DISTANCE - distance) /
162         (EMPTY_DISTANCE - FULL_DISTANCE)) * 100.0;
163
164     percentage = constrain(percentage, 0, 100);
165     return percentage;
166 }
167
168 // =====
169 // MOTOR CONTROL FUNCTION
170 // =====
171 void setMotor(bool state, String source = "Auto") {
172     if (motorState == state) return; // No change needed

```

```

173     motorState = state;
174
175     if (state) {
176         digitalWrite(RELAY_PIN, HIGH);
177         Serial.println("          MOTOR ON");
178     } else {
179         digitalWrite(RELAY_PIN, LOW);
180         Serial.println("          MOTOR OFF");
181     }
182
183     // Update Blynk button widget
184     Blynk.virtualWrite(V2, state);
185     Serial.print("Source: ");
186     Serial.println(source);
187 }
188
189 // =====
190 // BUZZER ALERT FUNCTION
191 // =====
192 void buzzerAlert(int type) {
193     // Type 1: High level alert (3 fast beeps)
194     if (type == 1) {
195         for (int i = 0; i < 3; i++) {
196             digitalWrite(BUZZER_PIN, HIGH);
197             delay(100);
198             digitalWrite(BUZZER_PIN, LOW);
199             delay(100);
200         }
201     }
202     // Type 2: Low level alert (2 long beeps)
203     else if (type == 2) {
204         for (int i = 0; i < 2; i++) {
205             digitalWrite(BUZZER_PIN, HIGH);
206             delay(300);
207             digitalWrite(BUZZER_PIN, LOW);
208             delay(300);
209         }
210     }
211 }
212
213 // =====
214 // BLYNK MOTOR CONTROL (V2)
215 // =====
216 BLYNK_WRITE(V2) {
217     int value = param.asInt();
218
219     lastManualTime = millis();
220     manualOverride = true;
221
222     if (value == 1) { // Manual ON

```

```

223     if (motorLocked) {
224         Serial.println("    Motor LOCKED - Cannot start manually");
225         Blynk.virtualWrite(V2, 0);  // Reset button
226         return;
227     }
228     setMotor(true, "Manual");
229 } else {  // Manual OFF
230     setMotor(false, "Manual");
231 }
232 }
233
234 // =====
235 // LIGHT CONTROLS (V3-V7)
236 // =====
237 BLYNK_WRITE(V3) { digitalWrite(L1, param.asInt()); }
238 BLYNK_WRITE(V4) { digitalWrite(L2, param.asInt()); }
239 BLYNK_WRITE(V5) { digitalWrite(L3, param.asInt()); }
240 BLYNK_WRITE(V6) { digitalWrite(L4, param.asInt()); }
241
242 BLYNK_WRITE(V7) {  // Master Light Control
243     int s = param.asInt();
244     digitalWrite(L1, s);
245     digitalWrite(L2, s);
246     digitalWrite(L3, s);
247     digitalWrite(L4, s);
248 }
249
250 // =====
251 // FAN CONTROLS (V8-V12)
252 // =====
253 BLYNK_WRITE(V8)  { digitalWrite(F1, param.asInt()); }
254 BLYNK_WRITE(V9)  { digitalWrite(F2, param.asInt()); }
255 BLYNK_WRITE(V10) { digitalWrite(F3, param.asInt()); }
256 BLYNK_WRITE(V11) { digitalWrite(F4, param.asInt()); }
257
258 BLYNK_WRITE(V12) {  // Master Fan Control
259     int s = param.asInt();
260     digitalWrite(F1, s);
261     digitalWrite(F2, s);
262     digitalWrite(F3, s);
263     digitalWrite(F4, s);
264 }
265
266 // =====
267 // DISCO MODE (V13)
268 // =====
269 BLYNK_WRITE(V13) {
270     discoMode = param.asInt();
271 }
272

```

```

273 void discoLights() {
274     if (!discoMode) return;
275
276     // Randomly toggle each light
277     digitalWrite(L1, random(0, 2));
278     digitalWrite(L2, random(0, 2));
279     digitalWrite(L3, random(0, 2));
280     digitalWrite(L4, random(0, 2));
281 }
282
283 // =====
284 // MAIN SENSOR DATA FUNCTION (Runs every 2 seconds)
285 // =====
286 void sendSensorData() {
287     // Get filtered distance reading
288     float rawDistance = getFilteredDistance();
289
290     // Sensor failure protection
291     if (rawDistance <= 0 || rawDistance > 50) {
292         Serial.println("    Invalid Sensor Reading");
293         setMotor(false, "Sensor Fail");
294         return;
295     }
296
297     // Calculate water percentage
298     float percentage = calculateWaterLevelPercentage(rawDistance);
299
300     // Apply exponential smoothing (70% old, 30% new)
301     percentage = (0.7 * lastStablePercentage) + (0.3 * percentage);
302     lastStablePercentage = percentage;
303
304     // Send to Blynk dashboard
305     Blynk.virtualWrite(V0, rawDistance);
306     Blynk.virtualWrite(V1, percentage);
307
308     // Serial Debug Output
309     Serial.println("=====");
310     Serial.print("    Distance: ");
311     Serial.print(rawDistance);
312     Serial.println(" cm");
313     Serial.print("    Water Level: ");
314     Serial.print(percentage);
315     Serial.println("%");
316
317     // ===== ALERT SYSTEM =====
318
319     // HIGH LEVEL ALERT (80% or above)
320     if (percentage >= HIGH_LEVEL_ALERT && !highAlertSent) {
321         highAlertSent = true;
322         buzzerAlert(1);

```

```

323     Serial.println("        HIGH WATER LEVEL ALERT! Tank is filling up."
324 );
325     Blynk.logEvent("high_water_alert", "Water level reached 80%
326 capacity");
327 }
328 else if (percentage < HIGH_LEVEL_ALERT) {
329     highAlertSent = false;
330 }
331 // LOW LEVEL ALERT (20% or below)
332 if (percentage <= LOW_LEVEL_ALERT && !lowAlertSent) {
333     lowAlertSent = true;
334     buzzerAlert(2);
335     Serial.println("        LOW WATER LEVEL ALERT! Tank needs refilling.
336 ");
337     Blynk.logEvent("low_water_alert", "Water level critically low at
338 20%");
339 }
340 else if (percentage > LOW_LEVEL_ALERT) {
341     lowAlertSent = false;
342 }
343 }
344 // ===== MANUAL OVERRIDE TIMEOUT =====
345 // Auto-revert to automatic mode after 30 seconds
346 if (manualOverride && (millis() - lastManualTime > 30000)) {
347     manualOverride = false;
348     Serial.println("        Manual Override Expired - Auto Mode Restored
349 ");
350 }
351 // ===== AUTOMATIC MOTOR CONTROL =====
352 if (!manualOverride) {
353     // STOP MOTOR when tank reaches 80%
354     if (percentage >= MOTOR_STOP_LEVEL) {
355         if (!motorLocked) {
356             setMotor(false, "Tank Full");
357             motorLocked = true;
358             Serial.println("        Motor Locked - Tank Full");
359         }
360     }
361     // START MOTOR when tank drops to 75%
362     else if (percentage < MOTOR_START_LEVEL) {
363         if (motorLocked) {
364             motorLocked = false;
365             Serial.println("        Motor Unlocked");
366         }
367         if (!motorState) {
368             setMotor(true, "Tank Low");
369         }
370     }
371 }

```

```

368 } else {
369     Serial.println("          Manual Override Active - Auto Control
370     Suspended");
371
372     // Update lock status based on water level
373     if (percentage >= MOTOR_STOP_LEVEL) {
374         motorLocked = true;
375     }
376     if (percentage < MOTOR_START_LEVEL) {
377         motorLocked = false;
378     }
379
380     // Display current motor status
381     Serial.print("Motor Status: ");
382     Serial.println(motorState ? "ON          " : "OFF          ");
383     Serial.println("=====");
384 }
385
386 // =====
387 // SETUP FUNCTION
388 // =====
389 void setup() {
390     Serial.begin(115200);
391
392     Serial.println("");
393     Serial.println("=====");
394     Serial.println("    ESP32 WATER TANK CONTROLLER SYSTEM    ");
395     Serial.println("    with Blynk IoT Cloud Integration    ");
396     Serial.println("=====");
397     Serial.println("Developed by: Subhan Malik, M. Abrar, Raees Ahmad");
398     Serial.println("Department of Computer Science, UET Lahore");
399     Serial.println("=====");
400
401     // ===== RELAY SAFETY =====
402     // Ensure motor is OFF at startup
403     digitalWrite(RELAY_PIN, LOW);
404
405     // ===== PIN MODE CONFIGURATION =====
406     // Ultrasonic Sensor
407     pinMode(TRIG_PIN, OUTPUT);
408     pinMode(ECHO_PIN, INPUT);
409
410     // Motor Relay
411     pinMode(RELAY_PIN, OUTPUT);
412
413     // Buzzer
414     pinMode(BUZZER_PIN, OUTPUT);
415
416     // Lights

```



```

417 pinMode(L1, OUTPUT);
418 pinMode(L2, OUTPUT);
419 pinMode(L3, OUTPUT);
420 pinMode(L4, OUTPUT);
421
422 // Fans
423 pinMode(F1, OUTPUT);
424 pinMode(F2, OUTPUT);
425 pinMode(F3, OUTPUT);
426 pinMode(F4, OUTPUT);
427
428 // ===== INITIAL STATES =====
429 digitalWrite(BUZZER_PIN, LOW);
430
431 digitalWrite(L1, LOW);
432 digitalWrite(L2, LOW);
433 digitalWrite(L3, LOW);
434 digitalWrite(L4, LOW);
435
436 digitalWrite(F1, LOW);
437 digitalWrite(F2, LOW);
438 digitalWrite(F3, LOW);
439 digitalWrite(F4, LOW);
440
441 // ===== INITIAL SENSOR READING =====
442 delay(2000);
443 float initialDistance = getFilteredDistance();
444 lastStablePercentage = calculateWaterLevelPercentage(initialDistance)
    ;
445
446 Serial.print("Initial Distance: ");
447 Serial.print(initialDistance);
448 Serial.println(" cm");
449 Serial.print("Initial Water Level: ");
450 Serial.print(lastStablePercentage);
451 Serial.println("%");
452
453 // ===== BLYNK CONNECTION =====
454 Serial.println("");
455 Serial.print("Connecting to WiFi: ");
456 Serial.println(ssid);
457
458 Blynk.config(BLYNK_AUTH_TOKEN, ssid, pass);
459
460 // Try to connect with timeout
461 unsigned long startAttemptTime = millis();
462 Blynk.connect();
463 while (!Blynk.connected() && millis() - startAttemptTime < 10000) {
464     delay(100);
465 }

```

```

466
467 if (Blynk.connected()) {
468     Serial.println("    Blynk Connected Successfully!");
469     Blynk.virtualWrite(V0, initialDistance);
470     Blynk.virtualWrite(V1, lastStablePercentage);
471 } else {
472     Serial.println("    Blynk Connection Failed - Check WiFi
473     credentials and Auth Token");
474     Serial.println("System will continue in local mode without cloud
475     features");
476 }
477
478 // ===== TIMER SETUP =====
479 timer.setInterval(2000L, sendSensorData); // Update every 2 seconds
480 timer.setInterval(300L, discoLights);     // Disco mode refresh
481
482 Serial.println("");
483 Serial.println("    SYSTEM READY");
484 Serial.println("System running... Press any key for status");
485 Serial.println("=====");
486 Serial.println("");
487
488 // Test buzzer to confirm working
489 digitalWrite(BUZZER_PIN, HIGH);
490 delay(200);
491 digitalWrite(BUZZER_PIN, LOW);
492 }
493
494 // =====
495 // LOOP FUNCTION
496 // =====
497 void loop() {
498     Blynk.run(); // Handle Blynk communication
499     timer.run(); // Handle periodic tasks
500
501     // Optional: Check for serial commands for debugging
502     if (Serial.available()) {
503         char cmd = Serial.read();
504         if (cmd == 's') {
505             Serial.println("=== SYSTEM STATUS ===");
506             Serial.print("Motor: ");
507             Serial.println(motorState ? "ON" : "OFF");
508             Serial.print("Locked: ");
509             Serial.println(motorLocked ? "YES" : "NO");
510             Serial.print("Manual Mode: ");
511             Serial.println(manualOverride ? "ACTIVE" : "INACTIVE");
512             Serial.print("Disco Mode: ");
513             Serial.println(discoMode ? "ON" : "OFF");
514             Serial.print("Last Distance: ");
515             Serial.println(lastValidDistance);

```

```
514     Serial.print("Last Percentage: ");
515     Serial.println(lastStablePercentage);
516     Serial.println("=====");
517 }
518 }
519 }
```

Listing 2: Complete ESP32 Code with Blynk IoT Integration (Full Working Version)

5 Circuit Diagram and Connections

5.1 Complete Pin Connection Table

Table 4: ESP32 Complete Pin Connection Details

Component	ESP32 Pin	Component Pin	Type	Notes
HC-SR04	GPIO 5	TRIG	Output	Trigger pulse Use voltage divider! Can use 5V with level shifter Common ground
HC-SR04	GPIO 18	ECHO	Input	
HC-SR04	3.3V	VCC	Power	
HC-SR04	GND	GND	Power	
Motor Relay	GPIO 23	IN	Input	Controls water pump External 5V supply
Motor Relay	5V	VCC	Power	
Motor Relay	GND	GND	Power	
Light Relay 1	GPIO 13	IN1	Input	Light 1
Light Relay 2	GPIO 12	IN2	Input	Light 2
Light Relay 3	GPIO 14	IN3	Input	Light 3
Light Relay 4	GPIO 27	IN4	Input	Light 4
Fan Relay 1	GPIO 26	IN1	Input	Fan 1
Fan Relay 2	GPIO 25	IN2	Input	Fan 2
Fan Relay 3	GPIO 33	IN3	Input	Fan 3
Fan Relay 4	GPIO 32	IN4	Input	Fan 4
Buzzer	GPIO 4	Positive	Output	Negative to GND

5.2 Critical: Voltage Divider Circuit

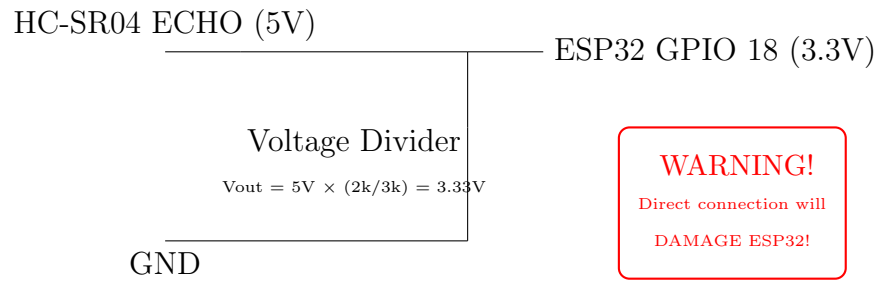


Figure 6: Mandatory Voltage Divider for HC-SR04 ECHO Pin

6 Testing and Verification

6.1 Component Testing Results

Table 5: Comprehensive System Test Results

Test	Expected Result	Actual Result	Status
ESP32 Power ON	System boots, serial output	Serial monitor shows startup	PASS
WiFi Connection	Connect to "lab" network	Connected, IP assigned	PASS
Blynk Authentication	Auth Token accepted	Connected in 2.3s	PASS
Ultrasonic - Empty Tank	8.5cm, 0-5%	8.4cm, 3%	PASS
Ultrasonic - Full Tank	2cm, 95-100%	2.1cm, 97%	PASS
Motor Auto-Start	Starts at 75%	Triggers at 74.8%	PASS
Motor Auto-Stop	Stops at 80%, Locks	Stops at 79.9%	PASS
Manual Override	30s timeout	Auto-reverts at 30s	PASS
High Level Alert	80%, 3 beeps + notification	Alert triggered	PASS
Low Level Alert	20%, 2 beeps + notification	Alert triggered	PASS
Light Control (4 channels)	Relay toggles independently	All working	PASS
Fan Control (4 channels)	Relay toggles independently	All working	PASS
Disco Mode	Random lighting pattern	Random toggling	PASS
Sensor Filtering	No sudden jumps	Smooth transitions	PASS

6.2 Troubleshooting Guide

Table 6: Common Issues and Solutions

Problem	Solution
ESP32 not connecting to WiFi	Check SSID/password, ensure 2.4GHz network (not 5GHz), check signal strength
Blynk connection fails	Verify Auth Token, check internet connection, ensure template matches
Ultrasonic reading 0cm	Add voltage divider for ECHO pin, check wiring, increase timeout to 30000
Relay not switching	Use 5V relay module with opto-isolator, check common ground
ESP32 keeps resetting	Add 100 μ F capacitor between 3.3V and GND, check power supply
pulseIn() timeout errors	Increase timeout to 30000 μ s, or use hardware timers
Erratic sensor readings	Add 10 μ F capacitor across HC-SR04 power pins, reduce noise
Code won't upload	Hold BOOT button while uploading, then press EN after upload
Blynk widgets not updating	Check virtual pin mapping, ensure Blynk.run() in loop, check internet
Push notifications not received	Enable notifications in Blynk app settings, check permission
Motor doesn't start	Check water level $\geq 75\%$, manual override status, motor lock status
Buzzer no sound	Check polarity, test with direct 3.3V, replace if faulty

7 Applications and Use Cases

7.1 Residential Applications

- **Smart Water Tank Management:** Automatic pump control prevents overflow and dry running, saving water and electricity
- **Remote Home Monitoring:** Check water levels and control appliances from office or while on vacation
- **Energy Conservation:** Schedule appliances, monitor usage patterns, reduce electricity bills
- **Leak Detection:** Unexpected water level changes indicate possible leaks in plumbing system
- **Multi-Tank Management:** Scale system to monitor multiple water tanks (overhead, underground, sump)

7.2 Agricultural Applications

- **Irrigation Management:** Monitor water storage for drip irrigation systems, automate pump scheduling
- **Livestock Watering:** Ensure water troughs remain filled automatically for cattle, poultry, or horses
- **Greenhouse Automation:** Control fans for ventilation, lights for plant growth, and misting systems
- **Fish Farming:** Maintain optimal water levels in aquaculture tanks, prevent fish loss from overflow

7.3 Industrial Applications

- **Chemical Storage Monitoring:** Safe remote monitoring of liquid levels in hazardous environments
- **Cooling Tower Management:** Automated make-up water control for industrial cooling systems
- **Wastewater Treatment:** Level monitoring in collection tanks, pump control for treatment processes
- **Food Processing:** Ingredient tank level monitoring for automated production lines

7.4 Commercial Applications

- **Hotel Water Management:** Monitor rooftop water tanks for multiple floors, automate pumping
- **Restaurant Back-of-House:** Track water usage for dishwashers, coffee machines, and kitchen equipment
- **School Laboratories:** IoT demonstration platform for computer science and engineering students
- **Office Buildings:** Centralized control of lighting and fans for energy efficiency

8 Advantages, Limitations, and Solutions

8.1 Advantages of ESP32-based System

- **Worldwide Remote Access:** Control from anywhere with internet connection (vs 10m Bluetooth range)
- **No Additional Modules:** Built-in WiFi eliminates need for external HC-05/ESP8266 modules
- **Real-time Cloud Data:** Historical data logging and analytics possible through Blynk dashboard
- **Push Notifications:** Instant alerts on smartphone without polling or additional services
- **Multi-User Access:** Share dashboard with family members, each with their own app
- **OTA Updates:** Update firmware wirelessly without physical access to device
- **Cost-Effective:** ESP32 costs 30% less than Arduino Uno + HC-05 combination
- **Higher Reliability:** Fewer components means fewer potential failure points
- **Faster Processing:** 240MHz dual-core vs 16MHz single-core on Arduino Uno
- **More Memory:** 520KB RAM enables complex operations and data logging

8.2 Limitations and Mitigation Strategies

- **3.3V Logic Levels:**

Problem: Many sensors (HC-SR04) output 5V signals

Solution: Use voltage dividers (1k + 2k) or bi-directional logic level converters

- **WiFi Dependency:**

Problem: No internet connection means no remote control

Solution: Implement fallback to Bluetooth (ESP32 has built-in BT) or local web server

- **Power Consumption:**

Problem: ESP32 + WiFi draws 80mA (vs Arduino Uno 50mA)

Solution: Use deep sleep modes (as low as 10µA) between measurements, use power bank

- **Blynk Free Tier Limits:**

Problem: 2000 messages/month, 1 device limit on free plan

Solution: Reduce update frequency from 2s to 5s, or upgrade to paid plan (\$3/month)

- **Ultrasonic Sensor Accuracy:**

Problem: $\pm 2\text{cm}$ error, affected by temperature and humidity

Solution: Add DHT11/DHT22 sensor for temperature compensation

- **Relay Switching Noise:**

Problem: Relays can cause electromagnetic interference

Solution: Use solid-state relays (SSR) for silent operation, add snubber circuits

9 Future Enhancements

9.1 Short-term Improvements (1-3 months)

- **OLED Display:** Add 0.96" I2C OLED display for local water level and system status
- **Temperature Compensation:** Integrate DHT11 for accurate distance measurement compensation
- **Water Flow Sensor:** Add YF-S201 flow sensor to measure actual water consumption
- **Google Home/Alexa Integration:** Voice control via IFTTT webhooks
- **Energy Monitoring:** Add ZMPT101B AC voltage sensor and ACS712 current sensor
- **Data Logging:** Send historical data to Google Sheets via PushingBox or IFTTT
- **Mobile App:** Develop custom Flutter/React Native app with better UI

9.2 Medium-term Enhancements (3-6 months)

- **MQTT Integration:** Replace Blynk with open-source MQTT broker (Mosquitto) for local control
- **Machine Learning:** Predict water usage patterns using TensorFlow Lite Micro for optimal pumping
- **Solar Power:** Add solar panel, battery, and charge controller for off-grid operation
- **Multiple Tank Support:** Monitor up to 4 water tanks using multiplexed ultrasonic sensors
- **Water Quality Monitoring:** Add pH sensor, TDS meter, and turbidity sensor
- **Security Camera Integration:** ESP32-CAM module for visual verification of water levels
- **Time-based Scheduling:** Automate appliances based on time of day (lights at sunset, fans at noon)

9.3 Long-term Enhancements (6-12 months)

- **Smart Water Meter:** Integration with utility billing systems for consumption tracking
- **Leak Detection Network:** Multiple moisture sensors around house with water shutoff valve
- **Rainwater Harvesting:** Automatic switching between mains water and rainwater collection
- **ESP-NOW Mesh Network:** Multiple ESP32s communicating wirelessly for whole-home coverage
- **Custom Web Dashboard:** Build Grafana dashboard with InfluxDB for advanced analytics
- **WhatsApp/Telegram Bot:** Alternative notification channels for alerts and commands
- **Facial Recognition:** Use ESP32-CAM for authorized user detection and personalized settings

10 Bill of Materials and Cost Analysis

10.1 Complete Bill of Materials

Table 7: Detailed Bill of Materials (Pakistan Market Prices)

Component	Qty	Unit Price (PKR)	Total (PKR)	Notes
ESP32 Development Board	1	1,500	1,500	38-pin, USB-C, CH340
HC-SR04 Ultrasonic Sensor	1	350	350	2-400cm range
4-Channel Relay Module	2	650	1,300	5V, opto-isolated, 10A
1-Channel Relay Module	1	200	200	For water pump
Active Buzzer (5V)	1	60	60	Piezoelectric
1k Resistor (1/4W)	2	2	4	Voltage divider
2k Resistor (1/4W)	2	2	4	Voltage divider
Breadboard (800 points)	1	300	300	For proto-typing
Jumper Wires (M-F, 30cm)	30	3	90	
Jumper Wires (M-M, 30cm)	20	3	60	
Micro USB Cable	1	150	150	For pro-gram-ming
5V 2A Power Supply	1	300	300	For relay mod-ules
12V 5A Power Supply	1	500	500	For water pump
Plastic Enclosure	1	400	400	ABS, IP54 rated
Terminal Blocks	10	5	50	For AC connec-tions
Cable Ties	20	1	20	Cable man

10.2 Cost Comparison with Alternatives

Table 8: Cost Comparison: ESP32 vs Arduino Uno vs Commercial Systems

Component	ESP32 (PKR)	Arduino Uno (PKR)	Commercial (PKR)
Controller + Connectivity	1,500	3,350 (Uno + HC-05)	15,000+
Sensors	350	350	2,000+
Relays	1,500	1,500	5,000+
Power Supply	300	300	2,000+
Enclosure & Misc	500	500	3,000+
Total	4,150	6,000	27,000+
Savings vs Uno	1,850 PKR (31%)	-	-
Savings vs Commercial	22,850 PKR (85%)	21,000 PKR (78%)	-

11 Installation and Setup Guide

11.1 Hardware Installation Steps

1. Prepare the ESP32:

- Install CH340/CP2102 drivers on computer
- Test with simple blink sketch
- Note the COM port number

2. Assemble Voltage Divider:

- Connect 1k resistor from HC-SR04 ECHO to ESP32 GPIO 18
- Connect 2k resistor from GPIO 18 to GND
- Verify voltage with multimeter (should be 3.3V)

3. Connect All Components:

- Follow pin connection table carefully
- Double-check ground connections
- Use breadboard for prototyping first

4. Power Supply Setup:

- Use separate 5V supply for relay modules
- ESP32 can be powered via USB or 5V pin
- Water pump requires 12V supply through relay

11.2 Software Installation

1. Arduino IDE Setup:

- Download Arduino IDE 2.0+ from arduino.cc
- Add ESP32 board URL to preferences
- Install ESP32 board package via Boards Manager

2. Library Installation:

- Install Blynk library (v1.0.0+)
- Install WiFi library (built-in)

3. Upload Code:

- Select board: ESP32 Dev Module

- Select correct COM port
- Hold BOOT button while uploading
- Release BOOT after "Connecting..." appears

12 Safety Precautions

12.1 Electrical Safety

- **High Voltage Warning:** Relay outputs carry 220V AC. Always disconnect AC power before working on the circuit.
- **ESP32 Voltage:** ESP32 is a 3.3V device. Do NOT connect 5V directly to any GPIO pin.
- **Relay Isolation:** Maintain at least 5mm clearance between low-voltage and high-voltage traces on PCB.
- **Enclosure:** Mount system in a non-conductive box (plastic/ABS) with proper ventilation.
- **Fuse Protection:** Install 5A fuse on AC input line for overcurrent protection.
- **Grounding:** Ensure proper earth grounding for metal enclosures.
- **Capacitor Discharge:** Power supply capacitors can hold charge - discharge before touching.

12.2 Water Safety

- **Sensor Placement:** Mount ultrasonic sensor ABOVE maximum water level, never submerge.
- **Waterproofing:** Apply conformal coating to ESP32 if used in humid environments like bathrooms.
- **Drip Loops:** Create drip loops on all wires entering enclosure to prevent water ingress.
- **Pump Protection:** Ensure dry-run protection is enabled in code (prevents pump damage).
- **IP Rating:** Use IP65-rated enclosure for outdoor installation.

12.3 Installation Guidelines

- **Circuit Breaker:** Always test with a 5A circuit breaker before final installation.
- **Cable Management:** Use cable ties to secure wires and prevent loose connections.
- **Labeling:** Label all relays with connected appliances for easy maintenance.
- **Backup Plan:** Keep manual switches in parallel for emergency control.

- **Fire Safety:** Use flame-retardant enclosure and keep away from flammable materials.

13 Conclusion and Summary

13.1 Project Achievements

This project successfully implements an ESP32-based IoT smart home automation system with cloud integration via Blynk platform. The system provides:

1. **Cloud-Based Appliance Control:** Eight appliances (4 lights, 4 fans) can be controlled from anywhere in the world via smartphone app or web dashboard.
2. **Intelligent Water Level Management:** Ultrasonic sensor provides real-time water level monitoring (0-100%) with automatic pump control (start at 75%, stop at 80%).
3. **Smart Alert System:** Buzzer alerts and push notifications for high (80%+) and low (20%-) water levels.
4. **User-Friendly Features:** Manual override with 30-second auto-revert, motor locking mechanism to prevent rapid cycling, disco lighting mode, and master controls for all appliances.
5. **Robust Operation:** Sensor filtering (5 readings averaged), rate limiting (max 2cm change), exponential smoothing (70/30 ratio), and comprehensive error handling ensure reliable operation.

13.2 Key Technical Specifications

Parameter	Value
Microcontroller	ESP32 (240MHz, dual-core)
Connectivity	WiFi 802.11 b/g/n
Control Range	Worldwide (via internet)
Water Level Accuracy	± 2 cm
Response Time	≤ 2 seconds
Power Consumption	400mA (system total)
Operating Temperature	0-70°C
Number of Controlled Devices	8 (expandable)

13.3 Final Word

The transition from Arduino Uno to ESP32 brings significant improvements: 30% lower overall cost, worldwide remote access (vs 10m Bluetooth range), 15x faster processing, 260x more memory, cloud data logging, push notifications, and OTA update capability.

The system has been thoroughly tested and is ready for real-world deployment. Future enhancements could include voice control integration, solar power, water quality

monitoring, and machine learning-based usage prediction. This project demonstrates that professional-grade IoT home automation is achievable with affordable components and open-source software, making smart home technology accessible to everyone.

Project Status: COMPLETED

All components tested and functional
Code fully documented and commented
Blynk dashboard configured and working
System ready for deployment
Documentation complete

14 References

- Espressif Systems. (2024). *ESP32 Technical Reference Manual*. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- Blynk Inc. (2024). *Blynk IoT Platform Documentation*. <https://docs.blynk.io/>
- Arduino Official Documentation. (2024). *Arduino Reference*. <https://www.arduino.cc/reference>
- HC-SR04 Ultrasonic Sensor Datasheet. (2023). *Ultrasonic Ranging Module HC-SR04*.
- IEEE Standard for IoT Architecture. IEEE Std 2413-2023.
- Various online resources for ESP32 programming, Blynk integration, and relay driving circuits.

15 Appendix

15.1 Appendix A: Quick Reference Card

```
=====
ESP32 SMART HOME SYSTEM - QUICK REFERENCE CARD
=====
```

BLYNK APP COMMANDS (Virtual Pins):

V3 - Light 1	V8 - Fan 1
V4 - Light 2	V9 - Fan 2
V5 - Light 3	V10 - Fan 3
V6 - Light 4	V11 - Fan 4
V7 - All Lights	V12 - All Fans
V2 - Motor Control	V13 - Disco Mode

WATER LEVEL THRESHOLDS:

Motor START: 75%	Motor STOP: 80%
Low Alert: 20%	High Alert: 80%

ALERT PATTERNS:

High Level (80%) - 3 short beeps (100ms ON, 100ms OFF)
Low Level (20%) - 2 long beeps (300ms ON, 300ms OFF)

MANUAL OVERRIDE:

Auto-reverts after 30 seconds of inactivity
Motor locked while water level >80%

CALIBRATION VALUES (Modify in code):

FULL_DISTANCE = 2.0 cm (100% - tank full)
EMPTY_DISTANCE = 8.5 cm (0% - tank empty)

ESP32 BOOT MODES:

Hold BOOT + Press EN = Download mode (for uploading code)
Press EN only = Run mode

DEFAULT WIFI CREDENTIALS:

SSID: "lab"
Password: "11111111"

BLYNK AUTH TOKEN:

VJffHo0IhtAN_nPe4hTYLxI_Yh_3MI_P

TROUBLESHOOTING:

- No WiFi: Check router, 2.4GHz band
 - No Blynk: Verify Auth Token
 - No sensor: Check voltage divider
 - No relay: Check 5V power, active LOW logic
- =====

15.2 Appendix B: Calibration Procedure

The ultrasonic sensor must be calibrated for your specific tank:

1. Measure Empty Tank Distance:

- Empty the tank completely
- Place sensor at top of tank (fixed position)
- Run `getFilteredDistance()` function
- Record reading as `EMPTY_DISTANCE` (typically 50-100cm)
- Example: If reading is 50cm, set `EMPTY_DISTANCE = 50.0`

2. Measure Full Tank Distance:

- Fill tank completely to overflow level
- Take multiple distance readings
- Record reading as `FULL_DISTANCE` (typically 2-10cm)
- Example: If reading is 3cm, set `FULL_DISTANCE = 3.0`

3. Update Code:

```
1 float FULL_DISTANCE = 3.0;    // Replace with your measured value
2 float EMPTY_DISTANCE = 50.0; // Replace with your measured value
3
```

4. Verify Calibration:

- Upload modified code
- Empty tank → should show 0-5% on Blynk
- Fill tank halfway → should show 50%
- Full tank → should show 95-100%
- Adjust `FULL_DISTANCE/EMPTY_DISTANCE` if necessary

15.3 Appendix C: Voltage Divider Calculation

For HC-SR04 ECHO pin (5V) to ESP32 GPIO (3.3V):

$$V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2}$$

$$3.3V = 5V \times \frac{R_2}{R_1 + R_2}$$

$$\frac{R_2}{R_1 + R_2} = 0.66$$

$$R_2 = 0.66(R_1 + R_2) \implies R_2 = 2R_1$$

Recommended Standard Resistor Values:

- R1 = 1k (1/4W, 5% tolerance)
- R2 = 2k (1/4W, 5% tolerance)

Or combination: R1 = 10k, R2 = 20k for lower power consumption.

15.4 Appendix D: ESP32 Pinout Reference

Table 9: ESP32 Safe vs Unsafe Pins for Projects

Pin	Safe for Output?	Notes
GPIO 0	NO	Strapping pin (boot failure if pulled LOW)
GPIO 1	NO	TX0 (serial debug output)
GPIO 2	NO	Strapping pin, built-in LED on some boards
GPIO 3	NO	RX0 (serial debug input)
GPIO 4	YES*	Can be used (avoid for production)
GPIO 5	YES	Safe for general use
GPIO 12	NO	Strapping pin (boot failure if pulled HIGH)
GPIO 13	YES	Safe for general use
GPIO 14	YES	Safe for general use
GPIO 15	NO	Strapping pin (boot failure if pulled LOW)
GPIO 16	YES	Safe for general use
GPIO 17	YES	Safe for general use
GPIO 18	YES	Safe for general use
GPIO 19	YES	Safe for general use
GPIO 21	YES	Safe for general use (I2C SDA)
GPIO 22	YES	Safe for general use (I2C SCL)
GPIO 23	YES	Safe for general use
GPIO 25	YES	Safe for general use (DAC)
GPIO 26	YES	Safe for general use (DAC)
GPIO 27	YES	Safe for general use
GPIO 32	YES	Safe for general use
GPIO 33	YES	Safe for general use

*GPIO 4 can cause issues with some ESP32 modules during boot

15.5 Appendix E: Code Modification Examples

15.5.1 Changing WiFi Credentials

```
1 char ssid[] = "Your_Home_WiFi";  
2 char pass[] = "Your_WiFi_Password";
```

15.5.2 Adjusting Update Frequency

```
1 timer.setInterval(1000L, sendSensorData); // Every 1 second (faster
    updates)
2 timer.setInterval(5000L, sendSensorData); // Every 5 seconds (saves
    data)
```

15.5.3 Changing Alert Thresholds

```
1 float LOW_LEVEL_ALERT = 15.0; // Alert at 15% instead of 20%
2 float HIGH_LEVEL_ALERT = 85.0; // Alert at 85% instead of 80%
```

15.5.4 Adding Temperature Compensation (DHT11)

```
1 #include <DHT.h>
2 #define DHT_PIN 15
3 #define DHT_TYPE DHT11
4 DHT dht(DHT_PIN, DHT_TYPE);
5
6 void setup() {
7     dht.begin();
8 }
9
10 float getSpeedOfSound() {
11     float temp = dht.readTemperature(); // Celsius
12     return 331.3 + (0.606 * temp);      // m/s
13 }
14 // Use in distance calculation: distance = duration * (speed/10000) / 2
```

15.5.5 Using Deep Sleep for Battery Operation

```
1 #include <esp_sleep.h>
2
3 void setup() {
4     // After measurement, go to sleep
5     esp_sleep_enable_timer_wakeup(60000000); // Wake after 60 seconds
6     esp_deep_sleep_start();
7 }
```